

Saúl Alexander Sánchez Bautista

Introducción

En el desarrollo de software moderno y el diseño de APIs, **JSON (JavaScript Object Notation)** se ha consolidado como el estándar de facto para el intercambio de información debido a su ligereza, legibilidad humana y compatibilidad universal con prácticamente cualquier lenguaje de programación. Sin embargo, JSON es un formato puramente estructural; esto significa que, aunque garantiza que la sintaxis de un archivo sea correcta (es decir, que los corchetes y las comas están en su lugar), **carece de un mecanismo nativo para verificar la lógica y consistencia de los datos que transporta**.

Esto puede llevar a que un archivo json contenga información errónea, tipos de datos incorrectos o estructuras incompletas que pongan en riesgo la estabilidad del sistema. Para solucionar esta limitación y otros problemas futuros, la comunidad tecnológica recurre a herramientas externas como **JSON Schema**. Esta especificación actúa como un contrato o manual de reglas detallado que define con precisión qué campos son obligatorios, qué tipos de datos se permiten y qué formatos específicos deben cumplir, transformando un simple formato de texto en un flujo de información seguro, predecible y altamente confiable.

Así como has conocido cómo esta gran herramienta resuelve una limitación de este importante formato, más adelante conocerás qué es exactamente, que significa, cómo se compone, sus características, virtudes e incluso, un ejemplo ilustrativo sobre como se ve un archivo json.



1.- ¿Qué es JSON?

El JSON es un formato de texto ligero diseñado para el intercambio de datos. En pocas palabras, es la espina dorsal del desarrollo de apps modernas, especialmente las que se basan en la arquitectura cliente-servidor.

Aunque se deriva de la sintaxis de JavaScript para representar estructuras de datos de objetos, hoy es compatible y utilizado por prácticamente todos los lenguajes de programación modernos, como Python.

2.- ¿Cuál es el propósito de JSON?

Es transportar y almacenar datos de manera ligera y ordenada entre diferentes sistemas, es decir, sirve para almacenar y estructurar información de manera legible tanto para las personas como para las máquinas.

3.- Características principales de JSON.

Las características que hacen reconocible este formato son las siguientes:

- **Es un formato ligero:** Su tamaño de archivo es el ideal para la optimización del envío de datos por internet.
- **Es fácil de leer:** Su estructura es lúcido y claro tanto para las personas como para las computadoras.
- **Pares de clave-valor (o variable-valor):** Organiza la información relacionando un nombre (clave) con su dato (valor).
- **“Políglota”:** No pertenece a un solo lenguaje; funciona con casi todos los lenguajes de programación modernos.
- **Posee una estructura jerárquica:** Permite meter datos dentro de otros datos, creando listas y objetos agrupados.

4.- Ventajas y desventajas del uso de JSON.

De acuerdo a sus diversas características, le permite tener ventajas, como:

Simplicidad y legibilidad. JSON tiene una sintaxis muy fácil de leer y escribir tanto para humanos como para máquinas.

Ligero y eficiente. Es más compacto ya que no usa etiquetas de apertura y cierre como XML.

Soporta varios lenguajes de forma nativa.

Compatibilidad absoluta con JavaScript. Ya que fué diseñado directamente a partir de la sintaxis de los objetos de ese mismo lenguaje, es lógico que estos conecten en su totalidad.

Estructuras anidadas, Es decir, se pueden usar listas y datos complejos dentro de un mismo bloque.

La gran mayoria de servicios web modernos lo utilizan (Google, Twitter, GitHub, Spotify).

Pero, así como tiene ventajas, tiene limitaciones, carencias o desventajas como:

Los datos que soportan son limitados. String, number, boolean, null, array y object

Sin comentarios en un archivo json, lo cual puede dificultar la comunicación entre desarrolladores en el mismo archivo.

Redundancia si el archivo es muy grande. Los archivos con muchas claves como “string” suelen ser más pesados y redundantes que casi cualquier otro formato, por lo que si buscas configuraciones complejas no es recomendable.

Es menos seguro si no se valida. Requiere de herramientas como JSON schema para validar los datos.

5.- Diferencia entre JSON y XML.

JSON y XML son formatos utilizados para almacenar y transmitir datos, sin embargo, cada uno tiene sus propias ventajas y diferencias. JSON es una opción popular para aplicaciones que se benefician de un formato de datos simple y fácil de usar. XML (Extensible Markup Language) es un lenguaje de marcado de propósito general similar a JSON que permite estructuras de datos más complejas.

6.- Usos actuales de JSON en APIs, aplicaciones web, móviles y servicios web.

En APIs y servicios o aplicaciones web se suelen usar para enviar información entre un servidor y una aplicación, permiten que programas distintos compartan datos sin problemas y son la base principal de las APIs modernas.

Además son la base fundamental en bases de datos no relacionales o NoSQL ya que, programas como MongoDB usan una estructura similar, además ayudan a cambiar los datos con total facilidad sin riesgo a que se dañe el sistema.

7.- Incluir un ejemplo sencillo de un objeto JSON y explicar sus partes principales.

Un archivo .json se compone de varios elementos que pueden conformar su estructura, a continuación veremos un ejemplo donde destacaremos cada punto:

```
{(1)
  "tienda" (3): (5)"Electrónica"(4),(6)
  "productos": [(2)
    {
      "id" (3),: 1,
      "nombre":(5), "Televisor",(6)
      "precio": 450.00(4)
    },
    {
      "id": 2,
      "nombre (3)": (5) "Auriculares (4)",(6)
      "precio": 25.50
    }
  ]
}
```

1. **Llaves {}:** Delimitan un **objeto** JSON.
2. **Corchetes []**: Delimitan un **arreglo** (lista) de elementos.
3. **Clave (Key)**: El nombre de una propiedad. Debe ser una cadena de texto entre comillas dobles (ej. `"nombre"`).
4. **Valor (Value)**: El dato asociado a una clave. Puede ser un string, número, booleano, nulo, otro objeto o un arreglo.
5. **Dos puntos (`:`)**: Separan la clave de su valor.
6. **Coma (`,`)**: Separa los diferentes pares de clave-valor o elementos de un arreglo.

Conclusión

La adopción masiva de **JSON** en la industria del software no es una casualidad, sino el resultado directo de su diseño minimalista, el cual prioriza la velocidad de transmisión y la simplicidad en la lectura de datos. No obstante, su mayor virtud —la flexibilidad técnica— es también su principal desafío, ya que la **falta de validación nativa** delega la responsabilidad de la seguridad y coherencia de los datos por completo al entorno de desarrollo.

Adoptar JSON en proyectos modernos exige entender que el formato no trabaja solo. Para mitigar el riesgo de procesar información corrupta o inconsistente, es fundamental integrar capas de validación robustas mediante estándares como **JSON Schema** o librerías de control de tipos. Al combinar la agilidad intrínseca de JSON con un sistema externo de reglas estrictas, las organizaciones logran el equilibrio perfecto: un intercambio de datos **altamente eficiente, predecible y blindado contra errores lógicos**.

¡Gracias por leer, espero te haya servido de aprendizaje!

Bibliografía

- https://www.oracle.com/database/what-is-json/
- https://www.esic.edu/rethink/tecnologia/que-es-archivo-json-por-que-es-estandar-desarrollo-apps-c
- https://www.oracle.com/latam/database/what-is-json/
- https://www.tutorialesprogramacionya.com/jsonya/tema7.html
- https://aws.amazon.com/es/compare/the-difference-between-json-xml/
- https://aws.amazon.com/es/compare/the-difference-between-json-xml/